

VIT Bhopal University
Build Your Own Project - VITyarthi

Student Grade Management System

A command-line application built with Python and OOP

Course	Python Essentials
Student name	Saksham Sontakke
Registration no.	26BCE10842
Institution	VIT Bhopal
Project type	Individual project (CLI, Python standard library only)

Table of Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions & Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings & Key Takeaways
14. Future Enhancements
15. References

2. Introduction

Keeping track of marks is something every class has to do. In many places it is still done on paper or in a spreadsheet where the teacher has to work out averages, grades and ranks by hand. This is slow, and one wrong number changes a student's result.

For my Python Essentials project I built a **Student Grade Management System**. It is a menu-driven program that runs in the terminal. You can add students, enter marks for different subjects, and the program works out the letter grade, average, GPA and pass/fail result. It also ranks the whole class and prints a class summary.

The project uses the topics from the course: variables and data types, dictionaries and lists, functions, loops, exception handling, file handling, modules and packages, and object-oriented programming (classes, objects, class methods, inheritance for custom exceptions). No database and no external libraries are used. Marks are saved in a simple JSON file.

3. Problem Statement

Manual handling of student marks is time consuming and causes mistakes in calculating averages, grades and ranks. There is a need for a small, easy-to-run program that stores marks, validates what is typed in, applies one fixed grading scale, and shows clear reports.

Objectives

- Store student records and subject marks so they are still available after the program is closed.
- Reject wrong input (bad registration number, marks outside 0-100, empty names) with a clear message.
- Calculate letter grade, average, GPA and pass/fail automatically using one grading scale.
- Show a ranked list, individual report cards and a class summary.
- Keep the code modular, documented and tested so that it is easy to read and extend.

Scope and target users

The system is meant for teachers, class representatives or students who manage the marks of a small class from the command line. It is not a multi-user or web system.

4. Functional Requirements

The project has three major modules: student management, marks and grading, and reporting. All input comes from the keyboard menu and all output is printed to the terminal.

ID	Module	Requirement	Input	Output
FR1	Student management	Add a new student	Reg. no, name	Confirmation or error
FR2	Student management	Search students by part of the name	Text	Table of matches
FR3	Student management	Delete a student (with confirmation)	Reg. no, y/n	Confirmation
FR4	Marks and grading	Add or update marks for a subject	Reg. no, subject, marks	Saved message

ID	Module	Requirement	Input	Output
FR5	Marks and grading	Remove a subject from a student	Reg. no, subject	Confirmation
FR6	Marks and grading	Calculate grade, GPA, pass/fail	Stored marks	Values on report
FR7	Reporting	Show a report card for one student	Reg. no	Formatted card
FR8	Reporting	List all students ranked by average	-	Ranked table
FR9	Reporting	Show class summary and subject averages	-	Summary text
FR10	Data	Save/load data automatically as JSON	-	data/students.json

5. Non-functional Requirements

Category	Requirement	How it is met
Usability	Simple numbered menu, clear prompts and readable output tables	menu in main.py, formatting in report.py
Reliability	Program must not crash on wrong input or a damaged data file	custom exceptions, try/except in the menu loop and in FileStorage.load()
Maintainability	Each file has one job; easy to change the grading scale	7 modules, SCALE list in GradeCalculator
Error handling	Every error shows a short, understandable message	ValidationError, StudentNotFoundError, DuplicateStudentError
Logging	Important actions are written to a log file	logging module -> data/app.log
Performance / resource use	Fast for a class of a few hundred students, tiny memory use	in-memory dictionary keyed by reg. no (O(1) lookup)
Portability	Runs anywhere Python 3.8+ is installed, no extra packages	standard library only

6. System Architecture

The program is split into layers. The menu (main.py) never touches the file directly; it only talks to StudentManager. StudentManager uses the Student model, the GradeCalculator, the validators and FileStorage. This means the storage could later be replaced without changing the menu.

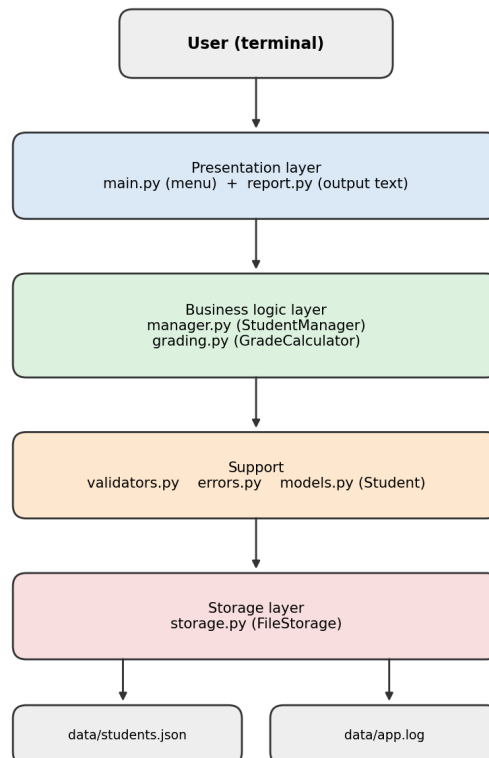


Figure 1: Layered system architecture

File	Responsibility
main.py	Menu loop, reads user input, calls the manager, prints errors
grade_manager/models.py	Student class (reg. no, name, marks, average, to/from dict)
grade_manager/grading.py	GradeCalculator: letter, grade point, GPA, pass/fail
grade_manager/manager.py	StudentManager: add, update, delete, search, rank, statistics
grade_manager/storage.py	FileStorage: load and save the JSON file
grade_manager/validators.py	Checks reg. no, name, subject and marks
grade_manager/errors.py	Custom exception classes
grade_manager/report.py	Builds report card, ranked table and class summary text
tests/	Unit tests for grading and manager logic

7. Design Diagrams

7.1 Use Case Diagram

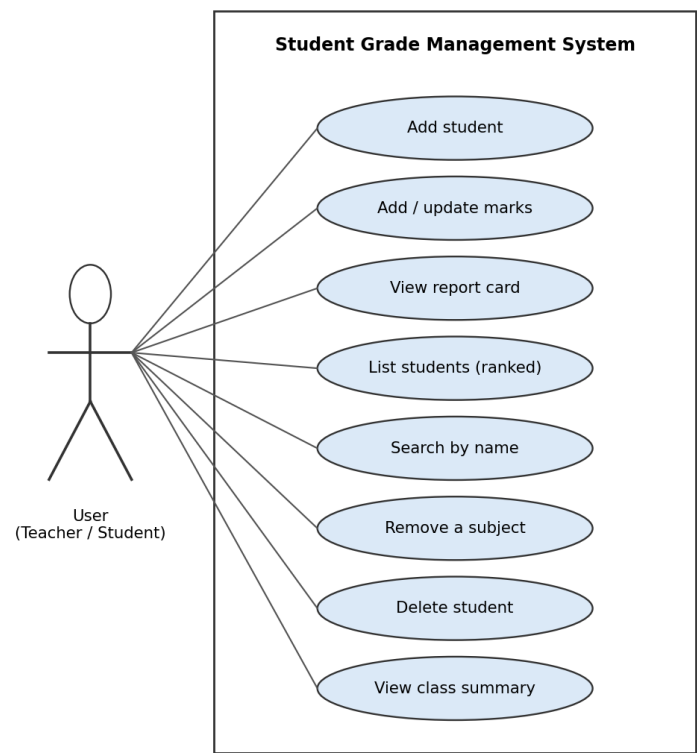


Figure 2: Use case diagram

7.2 Workflow Diagram

Every menu option follows the same flow: read the choice, collect the input, validate it, run the action, save the data and go back to the menu. Errors never stop the program.

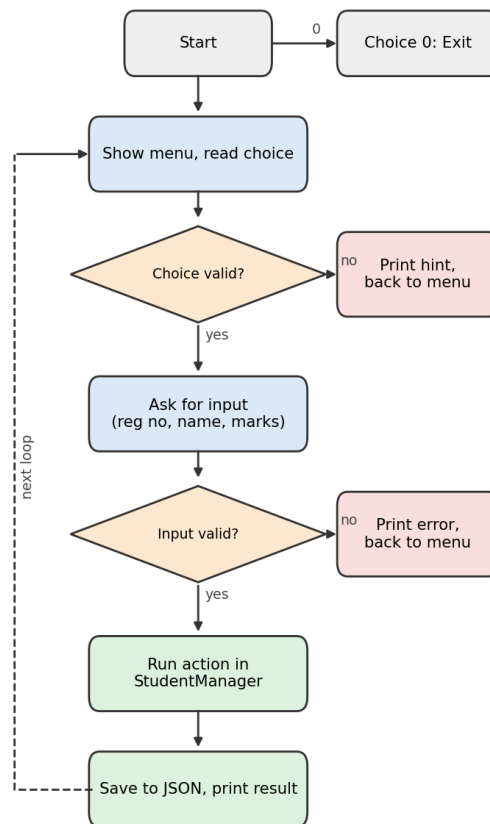


Figure 3: Workflow of one menu action

7.3 Sequence Diagram

The diagram shows what happens when the user adds marks for a student (option 2).

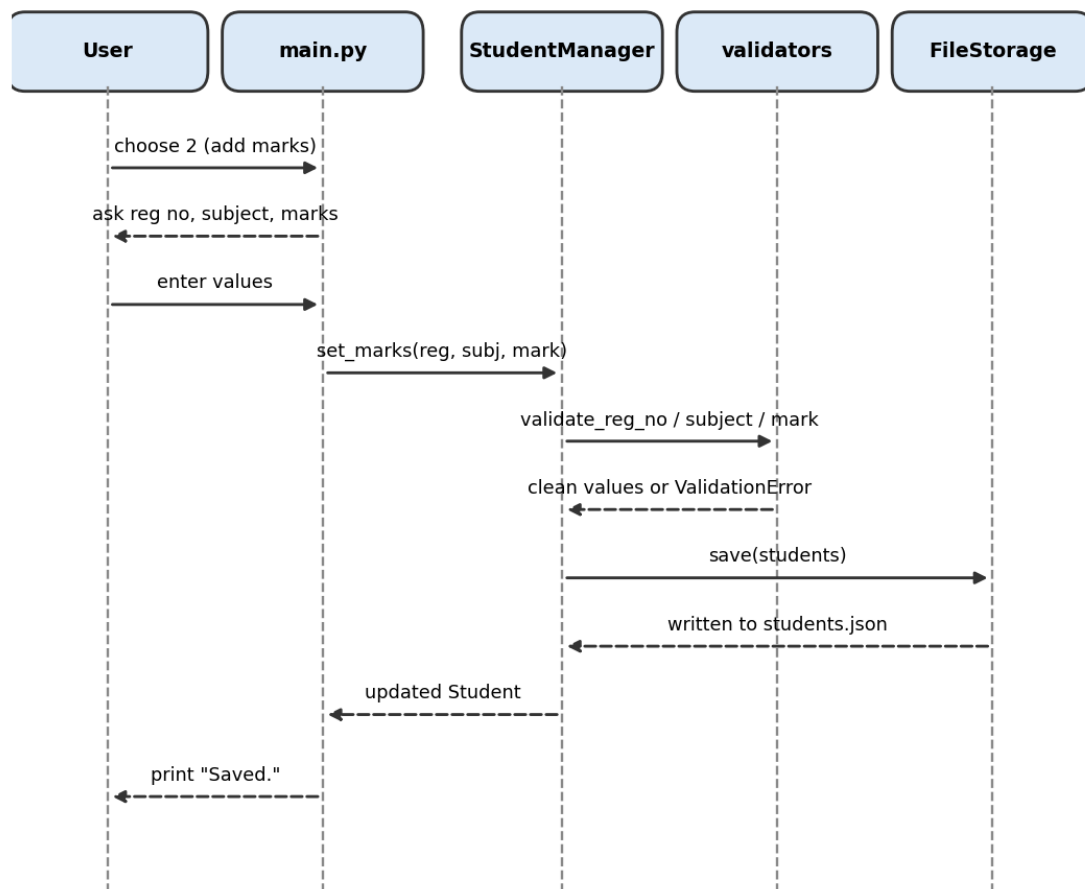


Figure 4: Sequence diagram for 'Add / update marks'

7.4 Class Diagram

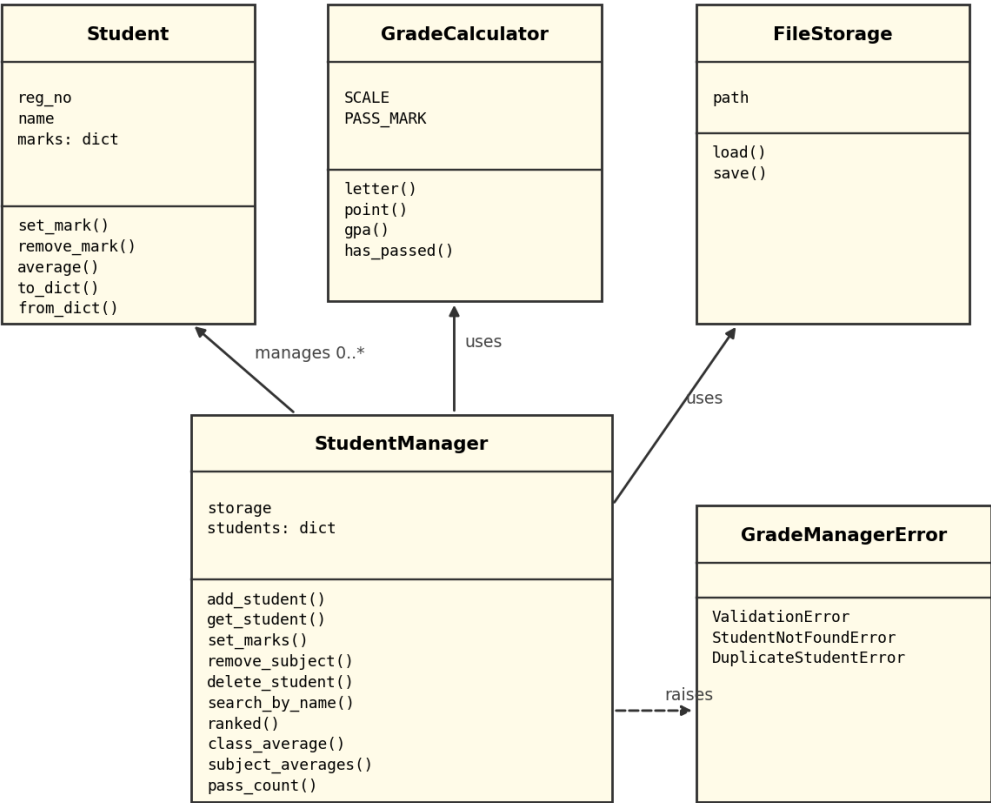


Figure 5: Class diagram

7.5 Data Model (ER Diagram)

The project uses a JSON file, not a database, so there are no tables. The data still has a clear structure: one student has many marks (one per subject). In the file the marks are stored inside the student record.

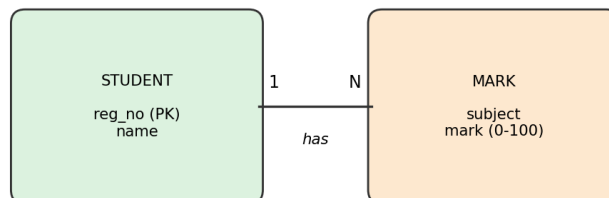


Figure 6: Entity relationship of the stored data

```
[
  {
    "reg_no": "26BCE10842",
    "name": "Saksham Sontakke",
    "marks": { "Python": 91.0, "Maths": 72.0 }
  }
]
```

8. Design Decisions & Rationale

Decision	Reason
JSON file instead of a database	The course covers file handling, not databases. JSON is readable and easy to check by hand.
Dictionary of students keyed by reg. no	Fast lookup and it prevents duplicate reg. numbers naturally.
Marks stored as a dictionary inside Student	Each student can have different subjects without a fixed number of columns.
Separate GradeCalculator class with class methods	Grading rules live in one place; no object is needed because it holds no per-student data.
Custom exception classes with a common parent	The menu can catch one base class and still print exact messages.
Validation in its own module	Same rules are reused by the manager and can be tested alone.
Save after every change	A crash or closed window does not lose data. Class sizes are small so cost is negligible.
Standard library only	Anyone can run it without installing packages.

9. Implementation Details

The project has about 500 lines of Python. Some key parts are explained below.

9.1 Student model (OOP)

Student keeps the registration number, the name and a dictionary of marks. It calculates its own average and can convert itself to a dictionary for saving and back again with a class method.

```
class Student:
    def __init__(self, reg_no, name, marks=None):
        self.reg_no = reg_no
        self.name = name
        self.marks = marks if marks is not None else {}

    def average(self):
        if not self.marks:
            return 0.0
        return sum(self.marks.values()) / len(self.marks)

    @classmethod
    def from_dict(cls, data):
        return cls(data['reg_no'], data['name'], dict(data.get('marks', {})))
```

9.2 Grading scale

Marks	Grade	Grade point
90 - 100	S	10
80 - 89	A	9
70 - 79	B	8
60 - 69	C	7
50 - 59	D	6
40 - 49	E	5
Below 40	F	0

GPA is the average of the grade points of all subjects. A student passes only when every subject mark is 40 or above.

9.3 Validation and error handling

Registration numbers are checked with a regular expression (two digits, three capital letters, five digits). Names may only contain letters and spaces. Marks must be a number from 0 to 100. Failed checks raise `ValidationError`. The menu loop in `main.py` catches `GradeManagerError`, prints the message and shows the menu again.

9.4 Storage and logging

`FileStorage.load()` returns an empty list if the file does not exist yet, and also if the JSON is damaged (a warning is printed and the error is logged). `save()` writes the whole dictionary back as indented JSON. Adds, updates and deletes are logged to `data/app.log` with a timestamp.

9.5 Version control

The project is kept in a Git repository on GitHub. The README explains setup, running and testing, and `statement.md` holds the problem statement, scope, target users and features.

10. Screenshots / Results

The screenshots below are the real output of the program, copied from a terminal session.

```

===== Student Grade Management System =====
1. Add student
2. Add / update marks
3. View report card
4. List students (ranked)
5. Search by name
6. Remove a subject
7. Delete student
8. Class summary
0. Exit
Choose an option: 1
Reg. no (e.g. 26BCE10842): 26BCE10842
Name: saksham sontakke
Added 26BCE10842 - Saksham Sontakke
Choose an option: 1
Reg. no (e.g. 26BCE10842): 26BCE10001
Name: asha verma
Added 26BCE10001 - Asha Verma
Choose an option: 1
Reg. no (e.g. 26BCE10842): 26BCE10002
Name: ravi kumar
Added 26BCE10002 - Ravi Kumar
Choose an option: 0
Goodbye!

```

Figure 7: Main menu and adding three students

```

Choose an option: 2
Reg. no: 26BCE10842
Subject (blank to stop): Python
Marks (0-100): 91
Saved.
Subject (blank to stop): Maths
Marks (0-100): 72
Saved.
Subject (blank to stop): Physics
Marks (0-100): 85
Saved.
Subject (blank to stop):
Choose an option: 2
Reg. no: 26BCE10001
Subject (blank to stop): Python
Marks (0-100): 64
Saved.
Subject (blank to stop): Maths
Marks (0-100): 38
Saved.
Subject (blank to stop):
Choose an option: 2
Reg. no: 26BCE10002
Subject (blank to stop): Python
Marks (0-100): 78
Saved.
Subject (blank to stop): Maths
Marks (0-100): 81
Saved.
Subject (blank to stop):
Choose an option: 4
Rank  Reg No      Name                Avg   GPA
-----
1      26BCE10842    Saksham Sontakke    82.67  9.00
2      26BCE10002    Ravi Kumar          79.50  8.50
3      26BCE10001    Asha Verma          51.00  3.50
Choose an option: 0
Goodbye!

```

Figure 8: Entering marks and the ranked student list

```

Choose an option: 3
Reg. no: 26BCE10842
=====
REPORT CARD: Saksham Sontakke (26BCE10842)
=====
Subject      Marks   Grade
-----
Maths        72.0    B
Physics      85.0    A
Python       91.0    S
-----
Average : 82.67
GPA      : 9.00
Result   : PASS
=====
Choose an option: 8
CLASS SUMMARY
-----
Students      : 3
Class average : 71.06
Passed        : 2 / 3
Subject averages:
  Maths        63.67
  Physics      85.00
  Python       77.67
Choose an option: 0
Goodbye!

```

Figure 9: Report card of a passing student

```

Choose an option: 3
Reg. no: 26BCE10001
=====
REPORT CARD: Asha Verma (26BCE10001)
=====
Subject          Marks   Grade
-----
Maths            38.0    F
Python           64.0    C
-----
Average : 51.00
GPA      : 3.50
Result   : FAIL
=====
Choose an option: 2
Reg. no: 26BCE10842
Subject (blank to stop): Python
Marks (0-100): 150
Error: Marks must be between 0 and 100.
Subject (blank to stop):
Choose an option: 1
Reg. no (e.g. 26BCE10842): 123
Name: abc
Error: Reg. no must look like 26BCE10842.
Choose an option: 5
Name contains: Ravi
Rank  Reg No      Name          Avg   GPA
-----
1    26BCE10002    Ravi Kumar      79.50  8.50
Choose an option: 0
Goodbye!

```

Figure 10: Failing report card, and error messages for invalid input

Results match hand calculation: Saksham Sontakke has marks 91, 72 and 85, so the average is 82.67 and the grade points are 10, 8 and 9, giving a GPA of 9.00. Asha Verma has 38 in Maths, so the result is FAIL even though her Python mark is 64.

11. Testing Approach

Testing was done in two ways: automated unit tests with Python's built-in `unittest` module, and manual runs through the menu with both valid and invalid input.

Test	What it checks	Expected result
test_letters	Marks 95, 80 and 39.9	S, A and F
test_gpa	Marks 95 and 75; empty marks	GPA 9.0; 0.0 when empty
test_pass_fail	40 vs 39 in one subject; no marks	pass, fail, fail
test_add_and_duplicate	Adding the same reg. no twice	DuplicateStudentError; name in title case
test_bad_input	Bad reg. no; marks 150	ValidationError
test_missing_student	Unknown reg. no	StudentNotFoundError
test_ranking_and_average	Two students, marks 60 and 90	Higher mark first; class average 75.0
test_data_survives_reload	Save then create a new manager	Marks are still there
test_delete	Delete a student	Student list is empty

The tests use a temporary folder, so they never touch the real data file. To run them use `python -m unittest discover -s tests -t ..`. All 9 tests pass.

Manual test cases

- Reg. no 123 is rejected with a clear message.
- Marks 150 and text such as abc are rejected.
- Choosing a menu number that does not exist prints a hint and shows the menu again.

- Deleting asks for confirmation; answering anything other than 'y' cancels.
- Deleting or corrupting `students.json` does not crash the program.

12. Challenges Faced

- **Deciding where validation belongs.** At first checks were mixed into the menu code. Moving them into `validators.py` made the manager reusable and the tests simpler.
- **Keeping the menu alive after errors.** Every action can raise an exception, so the loop needed one central `try/except` instead of many small ones.
- **Handling a damaged or missing data file.** The loader had to return an empty list instead of crashing on the first run or bad JSON.
- **Aligned output.** Getting tables and report cards to line up in the terminal needed format specifiers such as width and alignment.
- **Testing without touching real data.** The solution was to give `FileStorage` a path, so the tests can use a temporary file.

13. Learnings & Key Takeaways

- Splitting a program into small classes and modules makes it much easier to test and fix.
- Custom exceptions give cleaner error handling than returning error codes.
- Class methods (`@classmethod`) are useful for utility logic and alternative constructors.
- Writing tests early catches edge cases, such as an empty marks dictionary causing a division by zero.
- Good documentation (README, statement, comments) is part of the work, not an extra.

14. Future Enhancements

- Credit-weighted GPA (different subjects with different credits).
- Export reports to CSV or PDF.
- Edit a student's name and import many students from a CSV file.
- User login for teachers and read-only view for students.
- Move storage to a database once the course covers it, or add a simple GUI.

15. References

- Python Software Foundation, Python 3 documentation - <https://docs.python.org/3/>
- Python documentation: json, re, logging and unittest modules - <https://docs.python.org/3/library/>
- VITyarthi, Build Your Own Project instruction document
- Course material of Python Essentials, VIT Bhopal